

Colour Spaces

Three numbers, and which three you pick

You have already been clustering these

Last lecture ended by running k-means on the pixels of a painting and repainting each one with its centroid colour. It works. But it quietly assumed two things:

1. a pixel is a **point in a 3D space**;
2. **straight-line distance** in that space means “these colours are similar”.

Neither is obvious, and one of them is often wrong.

Predict: pixel value 255 is the brightest a channel gets. Is 128 about *half* as bright?

You have already been clustering these

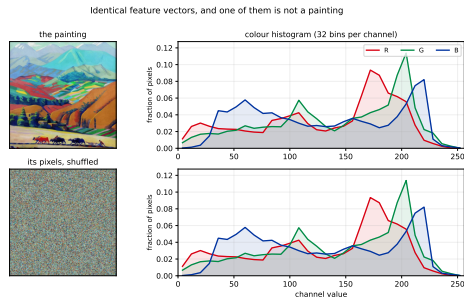
Last lecture ended by running k-means on the pixels of a painting and repainting each one with its centroid colour. It works. But it quietly assumed two things:

1. a pixel is a **point in a 3D space**;
2. **straight-line distance** in that space means “these colours are similar”.

Neither is obvious, and one of them is often wrong.

Predict: pixel value 255 is the brightest a channel gets. Is 128 about *half* as bright?

No. Pixel 128 emits **21.6%** of the light of 255. Half the light is pixel **188**.



Two images. One colour histogram.

Outline

What a pixel actually stores

Same pixel, different axes

Throwing colour away

Distance should mean difference

A pixel is not a colour

It is three numbers, and the reason there are exactly three has nothing to do with computers.

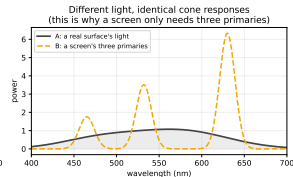
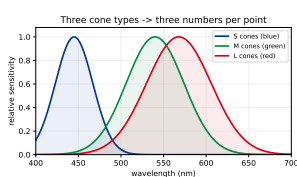
How your eye sees colour

Your retina has three kinds of **cone** cell, tuned to short, medium and long wavelengths - roughly **blue**, **green** and **red**.

Light arrives as a whole *spectrum*; your eye collapses it to **three numbers**. Everything else is thrown away.

Two spectra that fire the cones equally look identical to you - **metamerism**. A screen exploits exactly that: three primaries fake any spectrum you can see.

Images use **red, green, blue (RGB)** because *you* do. Three is a fact about eyes, not about file formats.



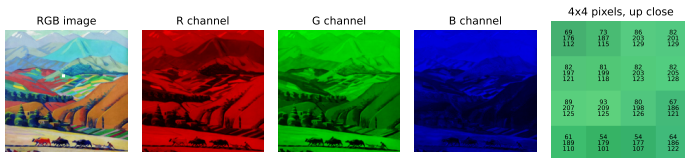
A pixel is three numbers

A colour image stores an **(R, G, B)** triple at every pixel, each 0 to 255.

Red and green high = yellow;
all three high = white; all three
equal = grey.

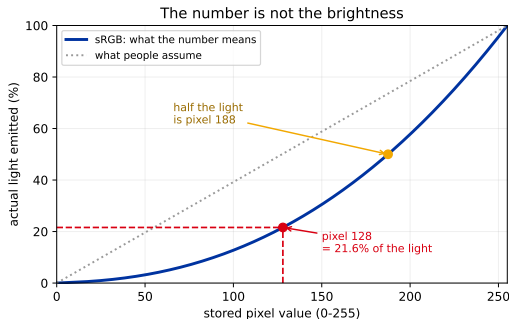
So an image is **three stacked grids**. Flatten them and you get the matrix you handed to k-means: one row per pixel, three columns.

To a computer there is no image, only a grid of numbers.

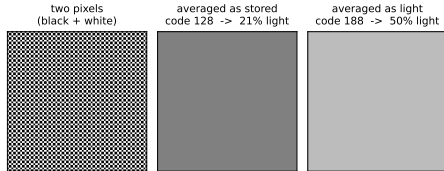


The number is not the brightness

Stored pixel values are **gamma-encoded**: the standard sRGB (standard RGB) curve spends more of the 0–255 budget on dark tones, where your eye can tell shades apart, and fewer on bright ones, where it cannot.



Shrinking an image the naive way loses light



Average black (0) and white (255) as *stored numbers* and you get 128 - which emits 21% of the light, not 50%. The honest answer is 188. Every naive image resize makes this mistake.

Linearize, compute, re-encode

The sRGB curve is a formula, not a mood. For a stored value s scaled to $[0, 1]$:

$$\text{linear}(s) = \begin{cases} s/12.92 & s \leq 0.04045 \\ \left(\frac{s + 0.055}{1.055}\right)^{2.4} & \text{otherwise} \end{cases} \quad \text{encode}(u) = 1.055 u^{1/2.4} - 0.055$$

So the claim from two slides ago, all the way through. Average black and white *as light*:

$$\text{linear}(0) = 0, \quad \text{linear}(1) = 1 \quad \Rightarrow \quad \text{mean} = 0.5 \quad \Rightarrow \quad 255 \cdot \left(1.055 \cdot 0.5^{1/2.4} - 0.055\right) = \mathbf{187.5}$$

And in the other direction, pixel 128: $\left(\frac{0.502+0.055}{1.055}\right)^{2.4} = \mathbf{0.216}$ - the 21.6% from the hook.

That is the whole of “linearize” and “re-encode” - one np.where each. The dark branch is a straight line because a pure power law has infinite slope at zero, which would make the darkest few values impossible to encode.

Where this bites you

Gamma is invisible until you do arithmetic on pixels. Then it is everywhere:

- ▶ **Resizing and blurring.** Both average neighbouring pixels. Done on stored values, fine detail loses light and gradients go muddy.
- ▶ **Brightness augmentation.** Multiplying stored values is not the same as changing the exposure of a photograph.
- ▶ **Averaging a dataset.** A mean image computed on stored values is darker than the mean of the light those images carried.
- ▶ **Any distance.** Two pixels 30 apart near black differ far more in light than two pixels 30 apart near white.

The fix is always the same three steps: **linearize** → do the arithmetic → **re-encode**.

Almost no code does this, including most of what you will write. That is usually survivable - but know that you are making the approximation, rather than discovering it later in a result you cannot explain.

RGB says how to make a colour

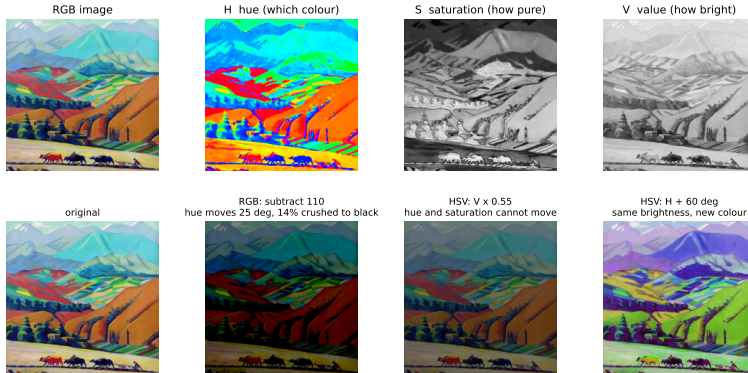
It does not say what the colour *is*. Nothing in the three numbers means “how bright” or “which colour”.

Play with it while I talk: math.hws.edu/graphicsbook/demos/c2/rgb-hsv.html

Six sliders, three RGB and three HSV, wired to the same colour. Move one, watch the others move.

Hue, saturation, value

Hue, saturation, value (HSV) keeps the same pixel and changes the axes: **hue** is an angle - which colour; **saturation** is a radius - how pure; **value** is a height - how bright.

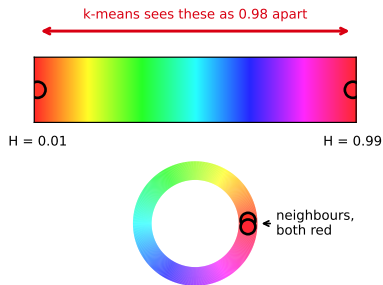


In RGB, “darker” is ambiguous: subtracting a constant drags the hue 25° and crushes 14% of pixels to black, while multiplying leaves hue alone. In HSV, brightness *is* an axis - moving V cannot do anything else.

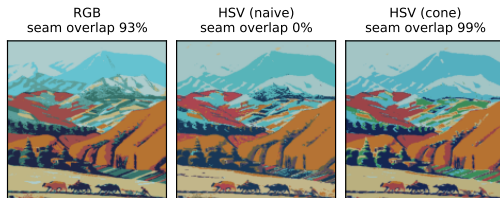
The trap: hue is a circle, distance is not

Hue wraps: $H = 0.99$ and $H = 0.01$ are both red. Hand those to any algorithm that measures straight-line distance and they are as far apart as two colours can be.

Hue wraps around; Euclid does not



k-means, $k = 8$: where the seam reds land



Feed k-means raw (H, S, V) and the reds either side of the seam land in the same cluster **0%** of the time. Encode the angle first - $(S \cos 2\pi H, S \sin 2\pi H, V)$ - and it is **99%**. Plain RGB, which has no angles in it, never had the problem.

When hue earns its keep

Pick out one coloured region, then change the lighting - every pixel loses 45% of its value, and nothing else moves.

The RGB threshold was tuned on the bright image and still collapses. The hue window barely notices: dimming moves V , not H .

Overlap before versus after, as Intersection over Union (IoU) - pixels both selections agree on, over pixels either one picked:

RGB **20%** hue **99%**



Colour masks and thresholds: use hue. It is the one job HSV is clearly best at.

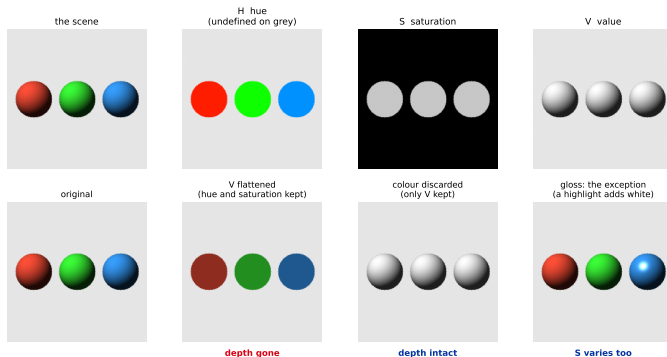
Where does depth live?

You read 3D shape from **shading**. So which of the three numbers carries it?

Where does depth live?

You read 3D shape from **shading**. So which of the three numbers carries it?

On a matte surface, changing the light **multiplies R, G and B by the same factor** - a move along V , with H and S untouched.



Across each lit sphere H and S hold to 10^{-9} ; V tracks the true shading at **1.000000**. Gloss is the exception - a highlight adds white, so it shows in S .

So: depth, edges and texture are luminance problems

That one fact settles a lot of practical questions at once:

- ▶ **Edge detection, corners, texture, optical flow** run on **one** channel, not three. Converting to grayscale first is not laziness, it is the right move - the structure was never in the colour.
- ▶ **Shape-from-shading and relighting** want a space where brightness is an axis. V , or better L^* , which is scaled to how you actually perceive brightness.
- ▶ **Chroma subsampling works for the same reason** - the slide two sections from now throws away three quarters of the colour and you will barely see it.
- ▶ **Colour is for what, luminance is for where.** Hue tells you which object; brightness tells you its shape and where its edges are.

The trap: dropping to grayscale also destroys any boundary that is a pure *colour* change at equal brightness - a red shape on an equally bright green background vanishes completely. Rare in photographs, common in charts, maps and synthetic images.

One number instead of three

Grayscale is the most common preprocessing step in vision, and the recipe is less obvious than it looks.

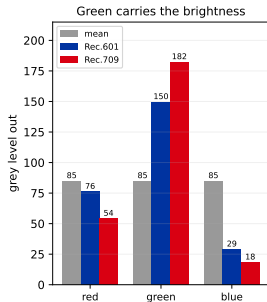
What should grey be?

Predict: to turn (R, G, B) into one number, is the average $(R + G + B)/3$ the right answer?

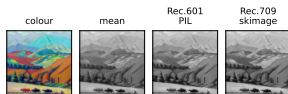
What should grey be?

Predict: to turn (R, G, B) into one number, is the average $(R + G + B)/3$ the right answer?

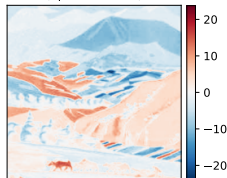
No - and the reason is the cone curves from the first section. The M and L cones both peak in the green-yellow region, so green carries most of the perceived brightness and blue very little.



Same painting, three recipes



Rec.601 minus Rec.709
(up to 24 levels)



Grayscale is a *weighted* sum, and the weights come from the eye: green about 60–72%, red about 21–30%, blue only 7–11%.

Two standards, and your libraries disagree

There are two weight sets in common use. Rec. 601 came from standard-definition television; Rec. 709 matches the primaries actually used by modern displays and sRGB.

	R	G	B
naive mean	0.333	0.333	0.333
Rec. 601	0.299	0.587	0.114
Rec. 709	0.2126	0.7152	0.0722

By hand, on the pixel $(R, G, B) = (200, 100, 50)$:

$$\text{Rec. 601: } 0.299(200) + 0.587(100) + 0.114(50) = 59.8 + 58.7 + 5.7 = \mathbf{124.2}$$

$$\text{Rec. 709: } 0.2126(200) + 0.7152(100) + 0.0722(50) = 42.5 + 71.5 + 3.6 = \mathbf{117.6}$$

`PIL.Image.convert("L")` implements Rec. 601. `skimage.color.rgb2gray` implements Rec. 709. On the Saryan painting they differ by **4.3** grey levels on average and up to **24**. Same input, two different grayscale images, and no error message.

Luma is not luminance

Both formulas above get applied to the **stored, gamma-encoded** numbers. That makes them *luma* (Y') - a cheap stand-in. Real *luminance* (Y) needs the light, not the code:

luma Y' (what everyone computes)	weight the stored values directly
luminance Y (what it should be)	linearize $\rightarrow$ weight $\rightarrow$ re-encode

On the same painting the two disagree by **4.1** levels on average and by as much as **48** in the worst pixel - a bigger error than the gap between the two standards.

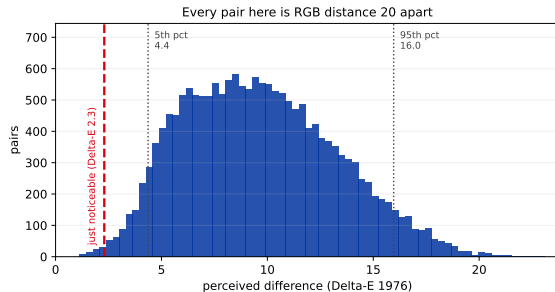
Video engineering has kept the distinction (and the prime on Y') for decades; image-processing code has mostly dropped it. If a grayscale result looks wrong in the shadows, this is the first thing to check.

The space you choose is the distance you get

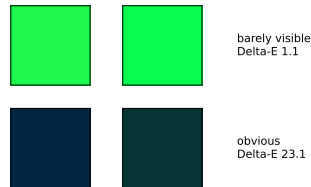
k-means has no idea what your features mean. It measures Euclidean distance in whatever you hand it.

Lab: built so that distance matches perception

CIELAB (from the Commission Internationale de l'Éclairage, the international colour standards body) splits colour into lightness L^* and two opponent axes a^* (green to red) and b^* (blue to yellow), scaled so that *equal distance means equal perceived difference*. That distance has a name: ΔE .



Same RGB distance, different worlds



Every pair above sits the *same* RGB distance apart, yet perceived difference runs from ΔE 4.4 to 16.0 - a $3.7\times$ spread, where about 2.3 is the threshold of noticing at all. RGB distance is a proxy, and a loose one.

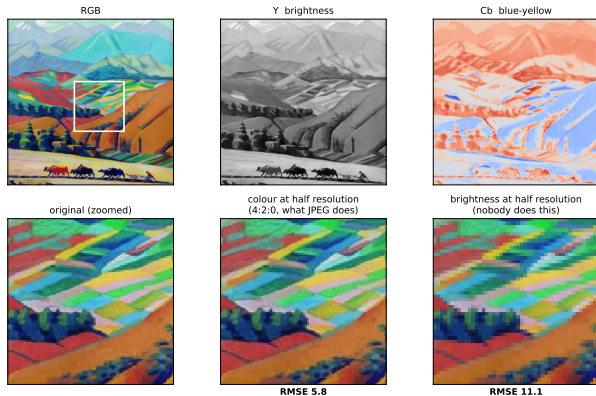
YCbCr: keep the brightness, skimp on the colour

Split a pixel into brightness Y and two colour-difference channels, C_b (blue-yellow) and C_r (red-green).

Store the colour channels at *half resolution* in each direction. That is **4:2:0** - what JPEG and every video codec do.

Halve the *brightness* detail instead and it visibly falls apart: error 5.8 versus 11.1, with blocky edges.

Your eye resolves *brightness* detail far better than *colour* detail. Compression has exploited that for thirty years.



Colour histograms: a feature that cannot see

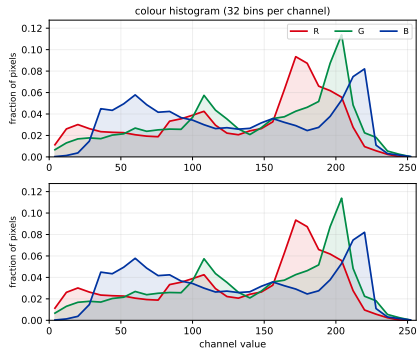
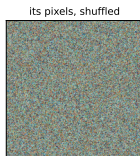
A **colour histogram** counts how many pixels fall in each range of each channel. It turns any image, at any size, into a fixed-length vector - here 96 numbers instead of 307,200.

Useful, with one fatal blind spot: it discards **where** every pixel was.

Shuffle the painting's pixels and the histogram does not move by a single count. Both images hand k-means the *identical* vector.

Cluster colour histograms and you group by **palette**, never by subject. A sunset and a tomato are neighbours.

Identical feature vectors, and one of them is not a painting



The wider zoo

CIE XYZ	The root (1931), straight from colour-matching experiments on human observers. Lab and everything else perceptual is <i>defined</i> as a transform of it.
HSL	HSV's sibling: lightness instead of value, so pure colours sit at $L = 0.5$ and white tops the axis. Nicer for pickers, same circular hue, same trap.
LCh	Lab in polar form - lightness, chroma, hue <i>angle</i> . Perceptual <i>and</i> intuitive, but the angle brings the seam back with it.
Oklab / Oklch	Björn Ottosson, 2020. Lab is non-uniform in the blues; Oklab refits it to fix that. In the CSS standard since 2021. The modern default.
CMYK	Subtractive - ink absorbs rather than emits, so the primaries are cyan, magenta, yellow and black. For print, not for models.
Linear RGB	Not a different colour, a different <i>encoding</i> : sRGB with the gamma undone. Where physics happens, and where you land whenever you linearize.

All the same three cone responses in different coordinates. Choosing one picks which questions become easy; it does not change what colour is.

Start from the task, not the space

What you are doing	Work in	Because
Edges, texture, depth, shading	grayscale / V / L^*	the structure is in luminance; colour adds nothing
Masking an object by its colour	HSV, hue window	survives lighting changes
Feeding a model; reconstructing	RGB	it is what the error is measured in, and what the data already is
Quantizing to get the image back	RGB	squared error lives here
Quantizing for a coherent palette	cone-encoded HSV	groups by hue across light and shade
"Do these two look different?"	Lab / Oklab, ΔE	distance is calibrated to perception
Compressing	YCbCr	spends bits where the eye looks
Any arithmetic on pixel values	linear RGB	sums are only meaningful on light

One question answers most of it: **is this about which colour, or about how much light?**
The first wants hue, the second wants a luminance axis.

Four ways to get it wrong

Each space has one failure that catches people, and you have now seen all four:

- ▶ **RGB** - distance only loosely matches perception, and every value is gamma-encoded, so arithmetic on it is arithmetic on the wrong quantity.
- ▶ **HSV** - hue is circular. Hand raw (H, S, V) to anything that measures distance and the reds tear in half.
- ▶ **Lab** - costs a conversion, and ΔE is calibrated for flat patches viewed side by side, not for textures or thin lines.
- ▶ **Grayscale** - two libraries will hand you two different images, and any boundary that is pure colour at constant brightness disappears entirely.

And three numbers is a convention, not a law. The Sentinel-2 satellite scene in the next practical gives you **six** per pixel, two of them infrared - light your eyes cannot see at all, which is exactly why it is worth recording. Same idea, more cones.

Recap

- ▶ A pixel is **three numbers** because you have **three cone types**. Different spectra that fire them equally are the same colour to you.
- ▶ Stored values are **gamma-encoded**: 128 is 21% of the light, not 50%. Arithmetic on pixels needs linearize → compute → re-encode.
- ▶ **HSV** makes brightness an axis, which is why hue thresholds survive lighting changes (99% versus 20%). But hue is **circular** - encode the angle or the seam destroys your clustering.
- ▶ **Grayscale** is a weighted sum with the weights taken from the eye. Two standards are in circulation and your libraries pick different ones.
- ▶ **Depth, edges and texture are luminance problems**. Shading is exactly a move along V , so grayscale keeps the shape and flattening V destroys it.
- ▶ **Lab** makes distance mean perceived difference; **YCbCr** lets compression throw colour away cheaply; **colour histograms** throw away position entirely.
- ▶ To pick a space, ask: **which colour, or how much light?**

Next: the practicals. Quantize a painting with k-means and choose the space deliberately; cluster a six-band satellite scene where the “colours” are land cover; and watch colour histograms fail to group photographs by what is in them.